

11、apriltag标签码跟随

11.1、简介

apriltag标签码追踪是在apriltag标签码识别的基础上，增加了小车车体运动的功能，控制车体从而使摄像头会保持标签码在视觉中间左右运动，在根据物体在摄像头成像近大远小的原理根据这一特性，从而实现标签码跟随功能。

11.2、启动

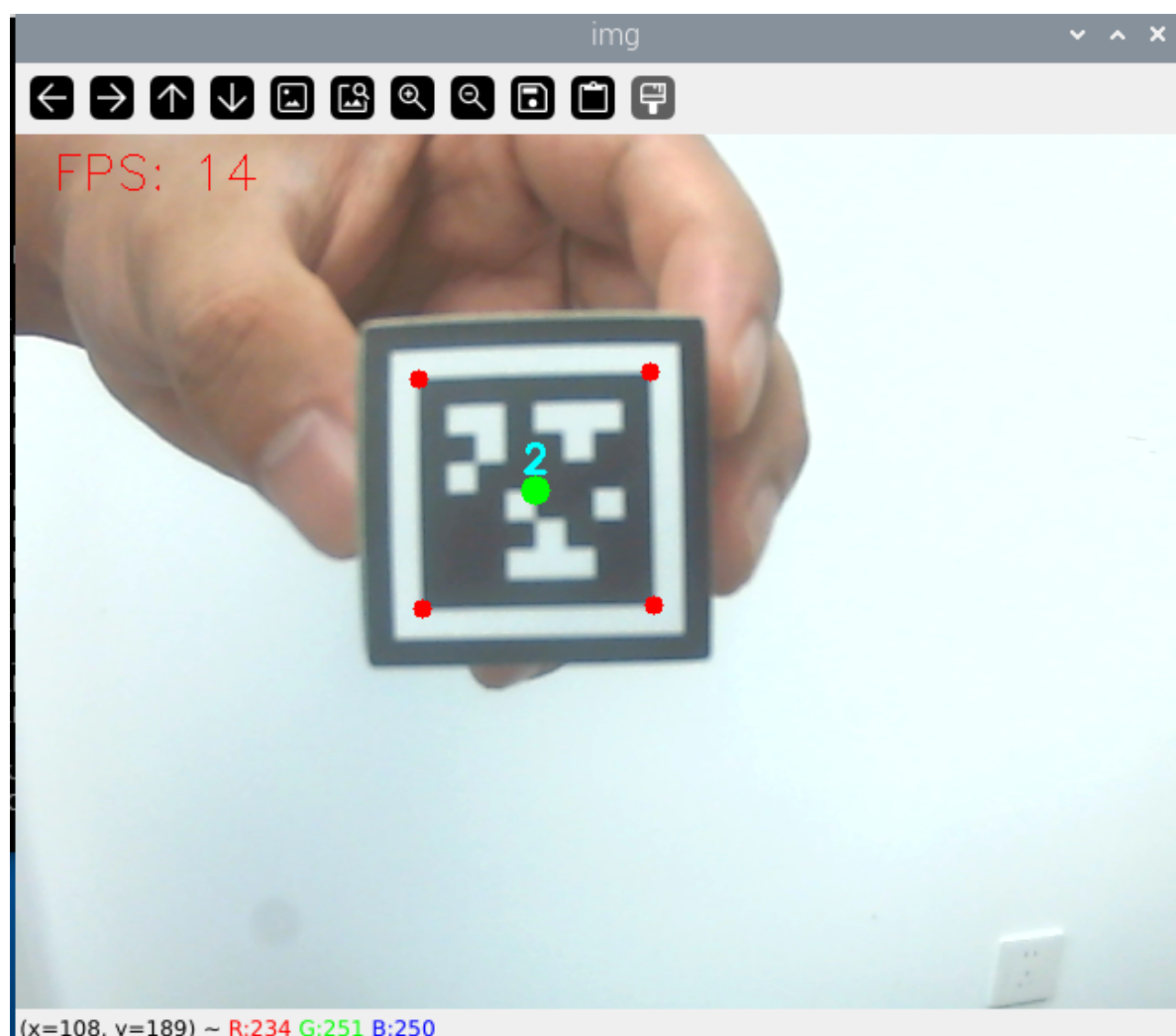
11.2.1、程序启动前的准备

本次apriltag标签码使用的是TAG36H11格式，出厂已配套相关标签码，并贴在积木块上，需要将积木块拿出来放置到摄像头画面识别。

11.2.2、程序说明

程序启动后，摄像头捕获到图像，将标签码放入摄像头画面，系统会识别并框出标签码的四个顶点，并显示标签码的ID号。然后缓慢移动积木块的位置，小车会跟着积木块前后移动、左旋右旋。

注意：积木块移动时，标签码要对着摄像头，并且移动速度不可以太快，避免摄像头云台跟不上。



11.2.3、程序启动

打开一个终端输入以下指令进入docker，

```
./docker_ros2.sh
```

出现以下界面就是进入docker成功

```
pi@yahboom:~ $ ./docker_ros2.sh
access control disabled, clients can connect from any host
root@yahboom:/#
```

在docker终端输入以下命令启动程序

```
ros2 launch yahboomcar_apriltag apriltag_follow.launch.py
```

11.2.4、源码

进入docker容器后，该功能源码的位置位于，

代码路

径：/root/yahboomcar_ws/src/yahboomcar_apriltag/yahboomcar_apriltag/apriltag_follow.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import yahboomcar_apriltag.fps as fps
import numpy as np
from yahboomcar_apriltag.vutils import draw_tags
from dt_apriltags import Detector
from yahboomcar_apriltag.PID import PositionalPID
from yahboomcar_apriltag.common import *
from Raspbot_Lib import Raspbot
import math

class TagFollowNode(Node):
    def __init__(self):
        super().__init__('tag_follow_node')
        self.twist = Twist()
        self.pub_cmdVel = self.create_publisher(Twist, "/cmd_vel", 1)
        self.h=0
        self.scale = 1000
        self.FollowLinePID = (60, 0, 20)
        self.linear = 0.05
        self.PID_init()
        # 初始化 Raspbot 实例
        self.bot = Raspbot()
        self.fps = fps.FPS() # 帧率统计器
        self.bridge = CvBridge()
        self.y servo_pid = PositionalPID(0.8, 0.6, 0.01) # PID控制器用于Y轴
        self.numx=self.numy=1
        target_servox = 90
        target_servoy = 25
        self.bot.Ctrl_Servo(1,target_servox)
        self.bot.Ctrl_Servo(2,target_servoy)
```

```

self.at_detector = Detector(searchpath=['apriltags'],
                             families='tag36h11',
                             nthreads=8,
                             quad_decimate=2.0,
                             quad_sigma=0.0,
                             refine_edges=1,
                             decode_sharpening=0.25,
                             debug=0)

self.subscription = self.create_subscription(
    Image,
    '/image_raw',
    self.image_callback,
    100)
self.subscription

def image_callback(self, ros_image):
    # cv_bridge
    try:
        cv_image = self.bridge.imgmsg_to_cv2(ros_image,
desired_encoding='bgr8')
    except Exception as e:
        self.get_logger().error(f"Failed to convert image: {e}")
        return

    # 使用 AprilTags 检测器
    tags = self.at_detector.detect(cv2.cvtColor(cv_image,
cv2.COLOR_BGR2GRAY), False, None, 0.025)
    tags = sorted(tags, key=lambda tag: tag.tag_id)

    # 绘制标签
    result_image = draw_tags(cv_image, tags, corners_color=(0, 0, 255),
center_color=(0, 255, 0))

    # 处理 AprilTags
    if len(tags) == 1:
        _, y, _, self.h = tags[0].bbox
        center_x =tags[0].center[0]
        if math.fabs(180 - (y + self.h/2)) > 20: #调试方块半径      Debug Block
Radius
            self.y servo_pid.SystemOutput = y + self.h/2
            self.y servo_pid.SetStepSignal(220)
            self.y servo_pid.SetInertiaTime(0.01, 0.1)
            target_valuey = int(650+self.y servo_pid.SystemOutput)
            target_servoy = int((target_valuey)/10)
            if target_servoy > 110:
                target_servoy = 110
            if target_servoy < 0:
                target_servoy = 0
            #self.bot.Ctrl_Servo(2, target_servoy)
            self.twist.linear.x = self.linear
            [z_Pid, _] = self.PID_controller.update([((center_x) - 320)*1.0/16,
0])

            self.twist.angular.z = +z_Pid
            #self.get_logger().info('center = {} angular =
{}'.format([(center_x)], [self.twist.angular.z]))
            #self.get_logger().info('h = {} '.format([h]))
            if(80<self.h<110):#调试目标半径70~90 Debug target radius 70~90

```

```

        self.twist.angular.z =0.0
        self.twist.linear.x= 0.0
    elif(self.h>110):#调试目标半径90 Debug target radius 90
        self.twist.linear.x= -self.linear
    elif(20<self.h<80):
        self.twist.linear.x= self.linear
    else:
        self.twist.angular.z =0.0
        self.twist.linear.x= 0.0

    else:
        self.twist.angular.z =0.0
        self.twist.linear.x= 0.0
        self.get_logger().info('h = {}linear = {} angular ={}'.format([self.h],
[ self.twist.linear.x],[self.twist.angular.z]))
        #print(self.twist.linear.x,self.twist.angular.z)
        self.pub_cmdvel.publish(self.twist)

    # 更新并显示 FPS
    self.fps.update()
    self.fps.show_fps(result_image)
    cv2.imshow("result_image", result_image)
    key = cv2.waitKey(1)
    if key != -1:
        cv2.destroyAllWindows()

def PID_init(self):
    self.PID_controller = simplePID(
        [0, 0],
        [self.FollowLinePID[0] / 1.0 / (self.scale), 0],
        [self.FollowLinePID[1] / 1.0 / (self.scale), 0],
        [self.FollowLinePID[2] / 1.0 / (self.scale), 0])

def main(args=None):
    rclpy.init(args=args)

    tag_follow_node = TagFollowNode()

    try:
        rclpy.spin(tag_follow_node)
    except KeyboardInterrupt:
        tag_follow_node.bot.Ctrl_Servo(1, 90)
        tag_follow_node.bot.Ctrl_Servo(2, 25)
    pass

if __name__ == '__main__':
    main()

```